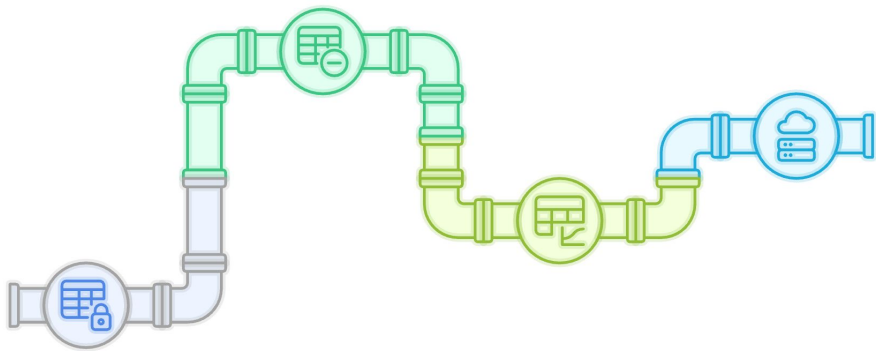




Serverless Data Pipelines and Observability

Jumanazar Gurbanov

October 2025

Workshop repo:

<https://github.com/juma-adoreme/bdw-dbt-template>

ADORE ME

Data Science at Adore Me



Home Try-On

Surprise boxes with personalization and diversity control.



Supply Chain

Demand forecasting and inventory optimization strategies.



Recommendation Systems

Recommendations for website and mobile app users.



Business Support

Supporting business teams in decision making processes.



Agenda



Serverless Data
Pipelines with
dbt, Prefect and
Cloud Run

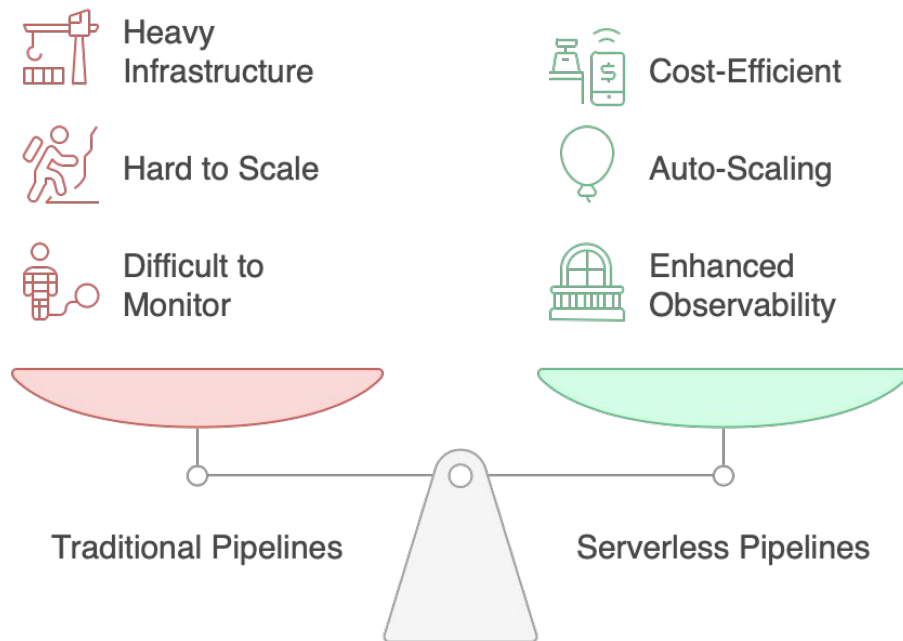


Data
Observability
with
Elementary



Q&A/Demo

Why Serverless Data Pipelines?



What is Cloud Run?

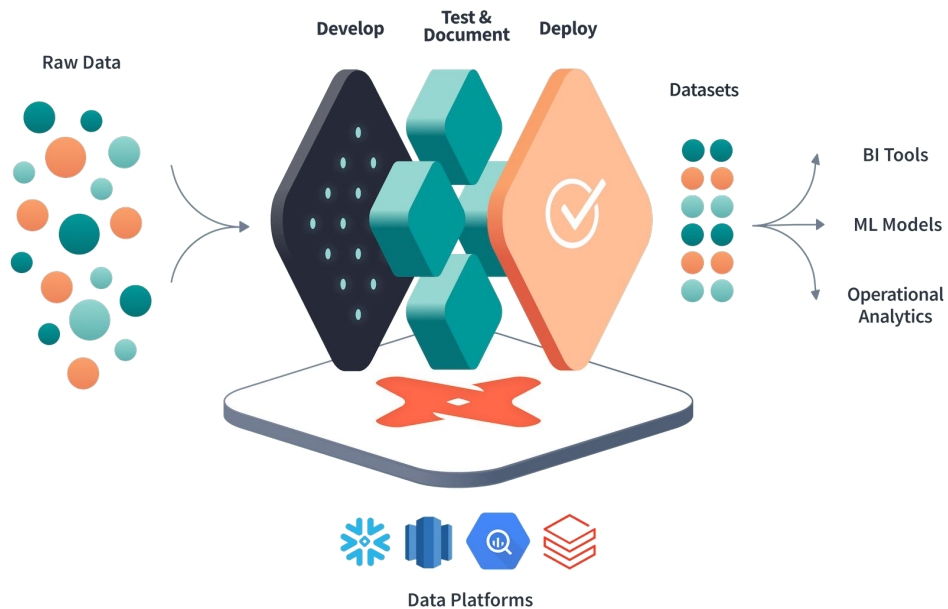
- **Serverless compute platform** by Google Cloud
- Runs **stateless containers** in a managed environment

Why to Deploy on Cloud Run?

- Cost efficiency (scales to zero)
- Trigger with Cloud Scheduler
- Minimal setup (just containerize your app)
- Pay-per-run model
- Quick Deployment and Iteration



What is DBT?



dbt (Data Build Tool)

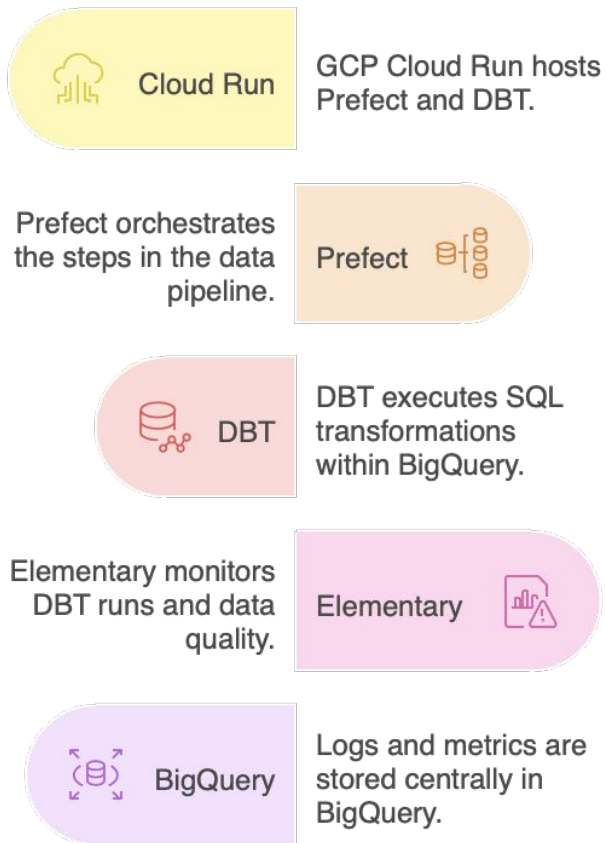
- SQL-based data transformation **framework**
- Declarative modeling: sources, **models**, tests
- Builds **modular**, version-controlled pipelines
- Generates **documentation** automatically

High-Level Architecture

Components:

- **Prefect** (orchestration)
- **DBT** (transformations in BigQuery)
- **Elementary** (observability)
- GCP **Cloud Run** (serverless compute)

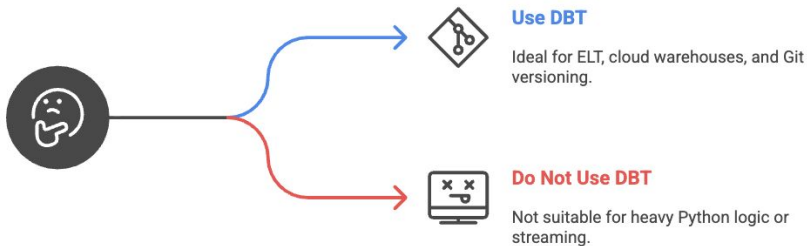
Data Pipeline Components



Who Uses DBT & When?

- **Used by:** Data Engineers, Analytics Engineers, Data Scientists
- **Best for:**
 - ELT (transform after load)
 - Track lineage & Docs
 - Modular, version-controlled transformation logic
 - Orchestrating SQL transformations

Should we use DBT for our data workflow?



Concepts:

- DAGs
- Models (Table/View)
- Sources
- Tests (Unit/Automated)
- Selectors
- Jinja

Commands:

- Run
- Compile
- Test
- List

DBT model example & compiled version

```
{{  
  config(  
    materialized = 'view',  
  )  
}}
```

```
SELECT  
  bq.customer_id  
  , bq.exclusion_amids  
FROM {{ ref('P_TD_BoxPreview_ExclusionList') }} bq  
LEFT JOIN {{ source('P_DM_Add_To_Box', 'P_TD_BoxPreview_ExclusionList_FirestoreState') }} fs  
ON bq.customer_id = fs.customer_id  
WHERE (  
  SELECT LOGICAL_OR(bq_exclusion_amids NOT IN UNNEST(fs.exclusion_amids))  
  FROM UNNEST(bq.exclusion_amids) bq_exclusion_amids  
)
```

```
SELECT  
  bq.customer_id  
  , bq.exclusion_amids  
FROM `am-elite-engines`.`P_DM_Add_To_Box`.`P_TD_BoxPreview_ExclusionList` bq  
LEFT JOIN `am-elite-engines`.`P_DM_Add_To_Box`.`P_TD_BoxPreview_ExclusionList_FirestoreState` fs  
ON bq.customer_id = fs.customer_id  
WHERE (  
  SELECT LOGICAL_OR(bq_exclusion_amids NOT IN UNNEST(fs.exclusion_amids))  
  FROM UNNEST(bq.exclusion_amids) bq_exclusion_amids  
)
```

Selectors & Operators

DBT models and tests can be compiled, ran, and listed by their

- Name, folder name
- Tags
- Materialization type (View/Table)

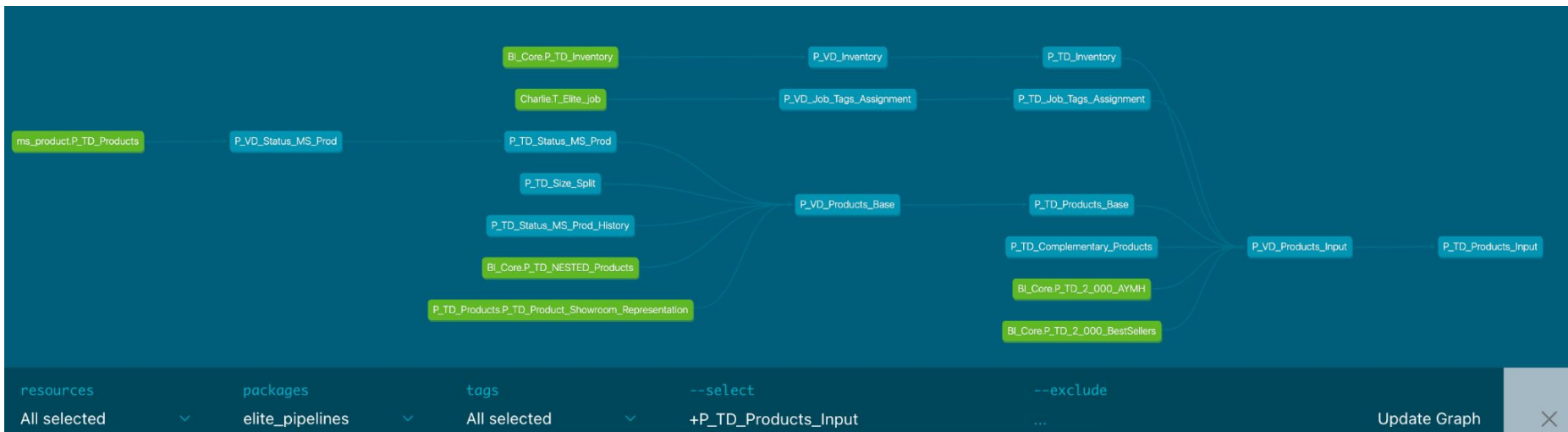
And all of this can be combined with

- Set operators: intersections & unions
- Graph Operators: +, n+, @

```
dbt run -m +P_TD_Products_Input,config.materialized:table --exclude tag:streaming -t prd
```

```
dbt run -m P_TD_BoxOrder_Margins P_TD_Order_Revenue -t prd
```

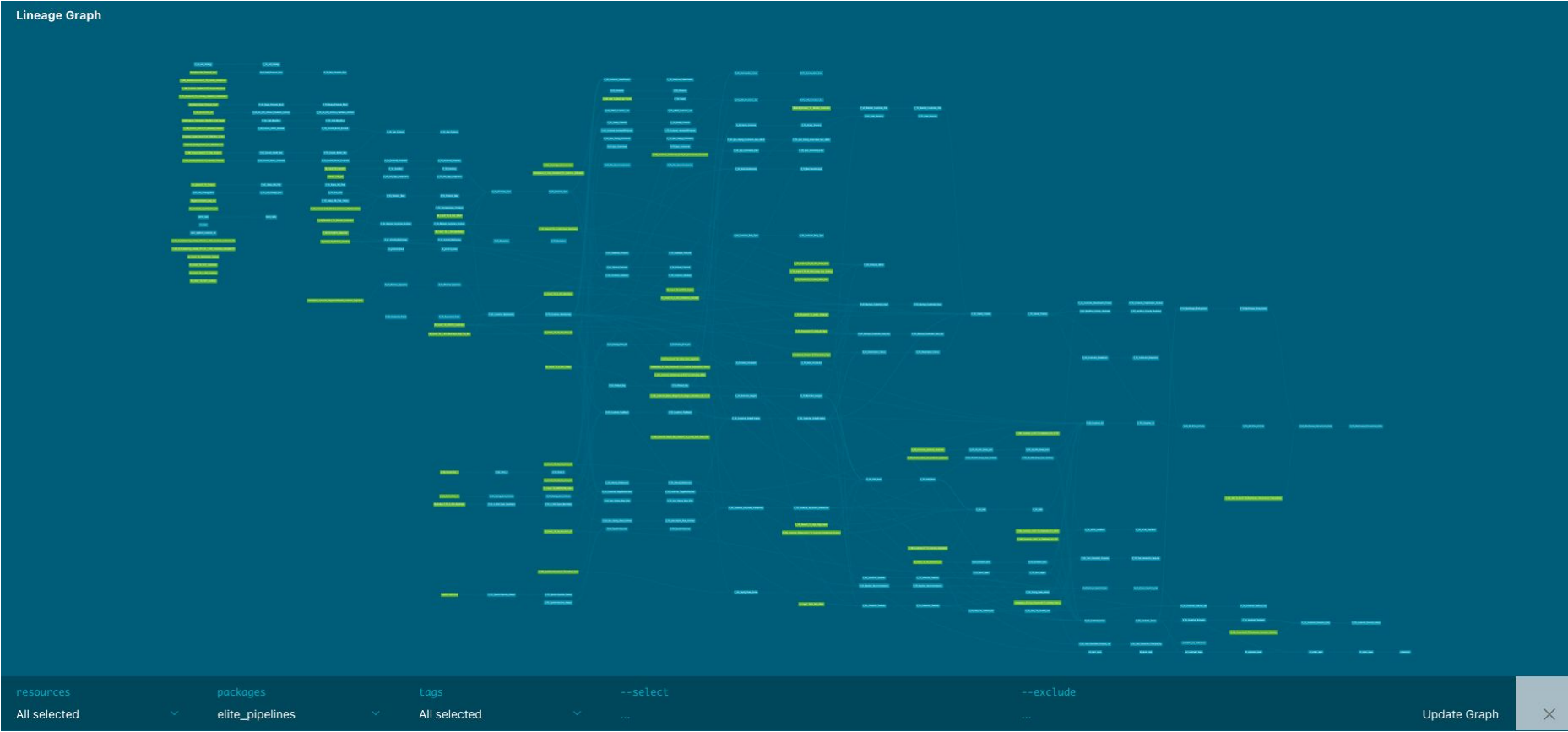
```
dbt ls -m +P_TD_Products_Input -t prd
```



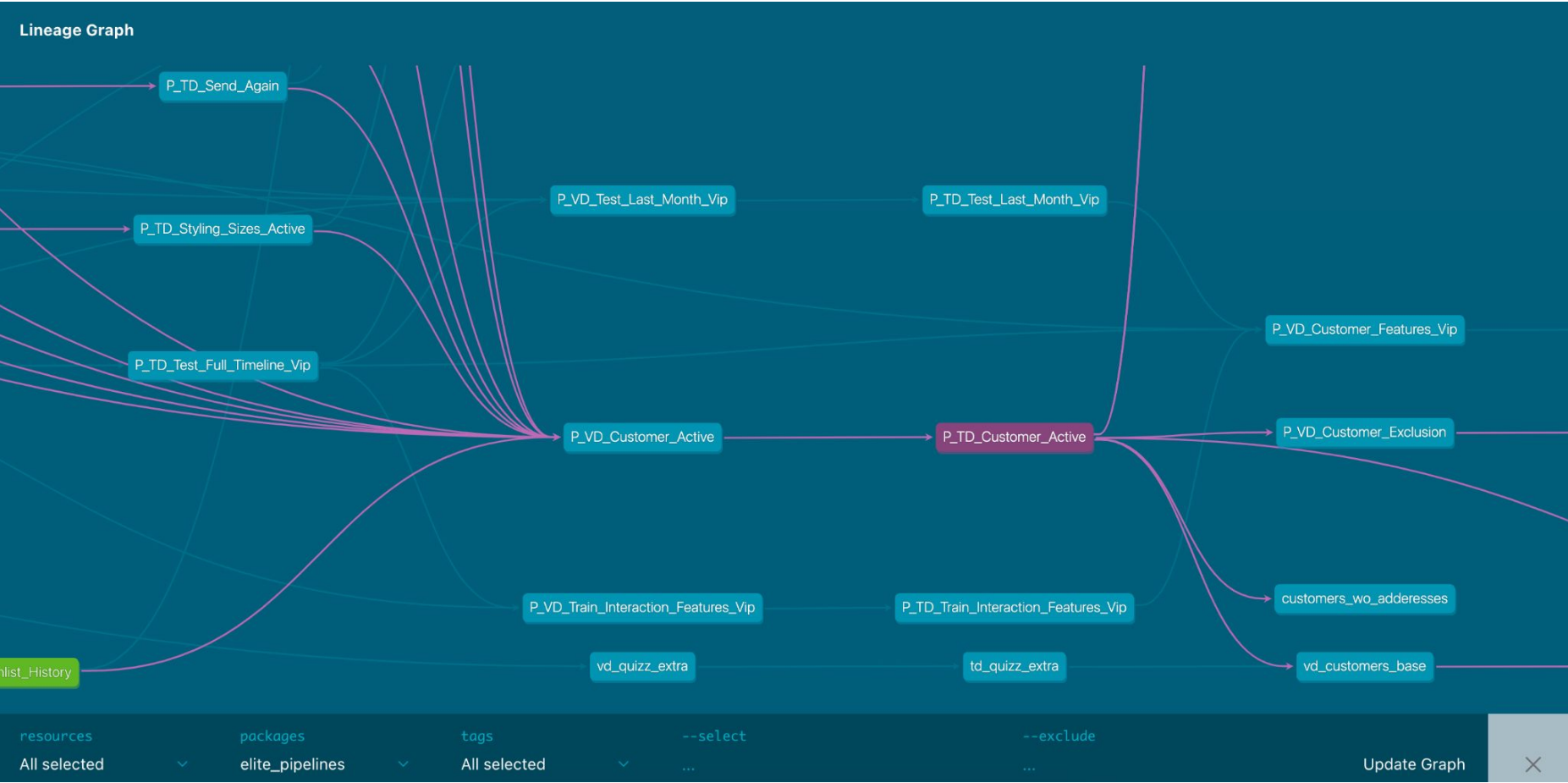
DBT Data Docs

Lineage Graph example

Green - sources (tables from DWH) Blue - DBT Models (View/Table), a processing step



Closer look into the chaos



Managing Environments

```
! profiles.yml
demo_project:
  outputs:
    dev:
      dataset: dev
      job_execution_timeout_seconds: 300
      job_retries: 1
      location: US
      method: oauth
      priority: interactive
      project: bigdataweek2025
      threads: 2
      type: bigquery
    prd:
      dataset: dbt
      job_execution_timeout_seconds: 300
      job_retries: 1
      location: US
      method: oauth
      priority: interactive
      project: bigdataweek2025
      threads: 2
      type: bigquery
  target: prd
```

```
pipes > dbt > macros > ❌ generate_schema_name.sql
1  {% macro generate_schema_name(custom_schema_name, node) -%}
2
3      {% set default_schema = target.schema -%}
4      {% if custom_schema_name is none -%}
5
6          {{ default_schema }}
7
8      {% elif target.name in ('prd', 'data_stg', 'stg') -%}
9
10         {{ custom_schema_name }}
11
12     {% else -%}
13
14         {{ default_schema }}_{{ custom_schema_name | trim }}
15
16     {% endif -%}
17
18 {% endmacro %}
```

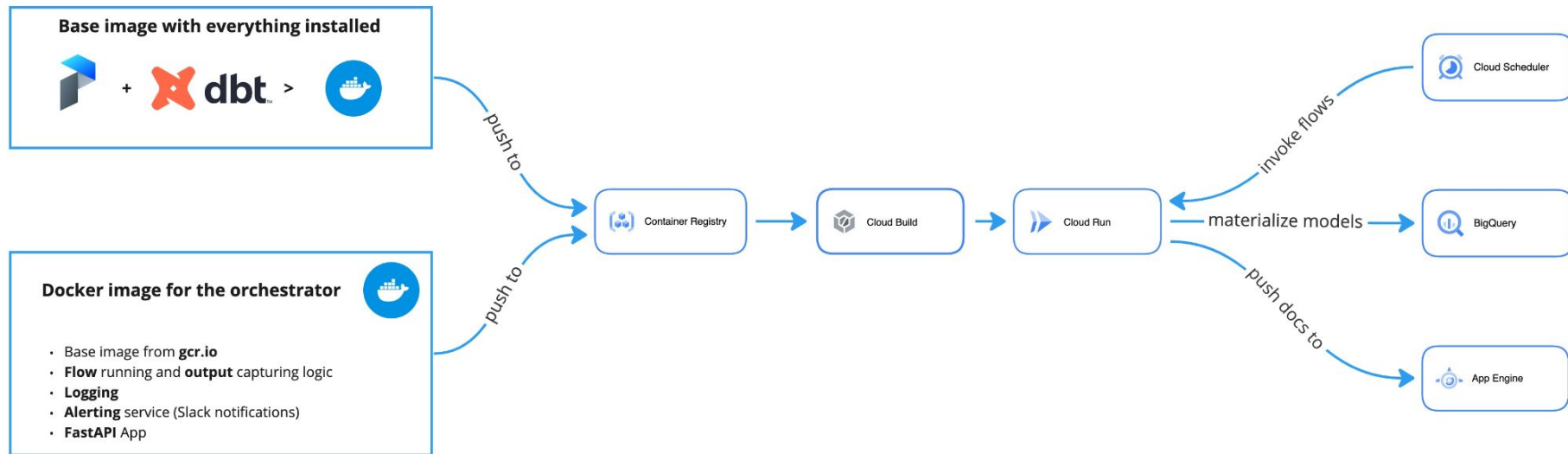
- Multiple Targets (prd, dev, etc)
- Multiple Threads
- Custom Schema/Object Naming

```
dbt ls --output json -m td_customers --output-keys database,schema,alias -t dev
{"database": "bigdataweek2025", "schema": "dev_dbt", "alias": "td_customers_base"}
```

```
dbt ls --output json -m td_customers --output-keys database,schema,alias -t prd
{"database": "bigdataweek2025", "schema": "dbt", "alias": "td_customers_base"}
```

Deployment Overview

- **How to deploy:** On Cloud Run, using Prefect as orchestrator
- **How it works:** Cloud Scheduler creates a request to the Cloud Run, running FastAPI app, which starts a Prefect Flow, which can execute the DBT DAG



Prefect Tasks & Flows

- Prefect is an open source orchestrator
- Each pipeline is a Prefect flow (multiple dependent tasks)

Prefect flow example consisting 2 DBT tasks:

1. DBT DAG - which runs all selected tables
2. DBT Test - data quality validation

```
run_dbt_active_customers_dag = DbtShellTask(
    command="dbt run -m +P_TD_Customer_Active,config.materialized:table --exclude tag:fraud tag:streaming
    helper_script=HELPER_SCRIPT,
    profiles_dir=DBT_PROFILES_DIR,
    profile_name=DBT_PROFILE_NAME,
    name="DBT Active Customers",
    return_all=True,
    log_stderr=True)

dbt_test_addresses = DbtShellTask(
    command="dbt test -m customers_wo_addresses -t prd",
    helper_script=HELPER_SCRIPT,
    profiles_dir=DBT_PROFILES_DIR,
    profile_name=DBT_PROFILE_NAME,
    name="Test customer addresses",
    return_all=True,
    log_stderr=True
)

flow_active_customers = Flow("Active Customers Flow")
flow_active_customers.add_task(run_dbt_active_customers_dag)
flow_active_customers.add_task(dbt_test_addresses)
dbt_test_addresses.set_upstream(run_dbt_active_customers_dag, flow=flow_active_customers)
```

FastAPI endpoints, API Contracts

How it works:

- **FastAPI** receives post request with a **flow name**
- Selector returns the flow, which is executed
- Outputs of the flow are captured and sent to the slack channel

FastAPI app

```
class PrefectRequestSchema(BaseModel):  
    flow_name: str  
    slack_channel_id: Optional[str] = "XXXXXXXXXX"
```

```
class ResponseSchema(BaseModel):  
    is_executed: bool  
    output: dict
```

```
app = FastAPI(openapi_url="/openapi.json")
```

```
@app.post("/prefect")
```

```
def run_flow(request: PrefectRequestSchema, response: Response):  
    channel_id = request.slack_channel_id  
    try:  
        flow_to_run = flow_selector(request.flow_name)  
        notify_pipeline_start(flow=flow_to_run, token=bot_token, channel_id=channel_id)  
        output = run_flow_and_return_output(flow_to_run)  
        notify_pipeline_end(response=output, token=bot_token, channel_id=channel_id)  
        response.status_code = status.HTTP_200_OK  
        results = {"is_executed": True, "output": output}  
        logging.info({"message": "Flow execution finished", **results})  
        return ResponseSchema(**results)  
    except Exception as e:  
        response.status_code = status.HTTP_500_INTERNAL_SERVER_ERROR  
        output = {"message": str(e)}  
        results = {"is_executed": False, "output": output}  
        logging.critical({"message": "Flow execution failed", **results})  
        return ResponseSchema(**results)
```

Flow selector

```
def flow_selector(flow_name: str):  
    if flow_name == "flow_active_customers":  
        from flows.active_customers import flow_active_customers  
        return flow_active_customers
```

Docker image entrypoint

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```


Monitoring/Slack Notifications

- Using Slack SDK in Python
- Flow output is parsed & formatted
- Pipeline starting & ending is notified
- Task outputs are attached to the message



PBot APP 11:00 AM

Active Customers Flow started and includes below tasks:

task_name	task_type	task_info
DBT Active Customers	DbtShellTask	dbt run -m +P_TD_Customer_Active,config.materialize tag:streaming -t prd
Test customer addresses	DbtShellTask	dbt test -m customers_wo_addresses -t prd



PBot APP 11:09 AM

Active Customers Flow is finished successfully 🎉

task_name	is_successful	message
DBT Active Customers	True	Task run succeeded.
Test customer addresses	True	Task run succeeded.

(edited)

2 files ▾



dbt output DBT Active Customers
Plain Text



dbt output Test customer addresses
Plain Text

```
def flow_output_slack(response: dict) -> str:
    message_text = tabulate(
        tabular_data=[
            {
                "task_name": wrap(task.get("task_name", ""), 50),
                "is_successful": task.get("is_successful", ""),
                "message": wrap(task.get("message", ""), 90)
            }
            for task in response.get("tasks")
        ],
        headers="keys",
        tablefmt="grid"
    )
    return message_text
```

```
def flow_info_slack(flow: Flow) -> str:
    tasks_info = tabulate(
        tabular_data=[
            {
                "task_name": wrap(task.name, 50),
                "task_type": re.search(r"(?<=\\.)(\\w+)(?=')", str(type(task))).group(1),
                "task_info": wrap(task.command if hasattr(task, "command") else (task.checkpoint if hasatt
            }
            for task in flow.tasks
        ],
        headers="keys",
        tablefmt="grid"
    )
    slack_message = f"*{flow.name}* started and includes below tasks:\n```${tasks_info}```"
    return slack_message
```

Monitoring/DBT Run History into Bigquery

```
1 {% macro run_logs_start(custom_alias_name=none, node=none) %}
2 {% if target.name in ['prd', 'data_stg', 'stg'] %}
3 {% set query %}
4 insert into `{{custom_alias_name}}.pipelines_run.objects_run` (
5     object_schema,
6     object_name,
7     run_environment,
8     start_time,
9     end_time,
10    status,
11    invocation_id)
12 values (
13     '{{this.schema}}',
14     '{{node.name}}',
15     '{{target.name}}',
16     current_timestamp,
17     null,
18     'started',
19     '{{invocation_id}}')
20 {% endset %}
21 {% do run_query(query) %}
22 {% endif %}
23 {% endmacro %}
```

```
1 {% macro run_logs_end(custom_alias_name=none, node=none) %}
2 {% if target.name in ['prd', 'data_stg', 'stg'] %}
3 {% set query %}
4 update `{{custom_alias_name}}.pipelines_run.objects_run`
5 set end_time = current_timestamp,
6     status = 'finished'
7 where invocation_id = '{{invocation_id}}'
8 and object_schema = '{{this.schema}}'
9 and object_name = '{{node.name}}'
10 {% endset %}
11 {% do run_query(query) %}
12 {% endif %}
13 {% endmacro %}
```

Row	object_schema	object_name	run_environment	start_time	end_time	status	invocation_id
146373	P_DM_Add_To_Box	P_TD_Products_Protected	prd	2023-06-28 12:01:06.030691 U...	2023-06-28 12:01:11.170563 U...	finished	f7bef889-47fc-41ed-aadd-eed6b39...
146374	P_DM_Add_To_Box	P_TD_Job_Strategy	prd	2023-06-28 09:00:41.219168 U...	2023-06-28 09:00:45.852748 U...	finished	ab20909d-d10b-432e-8094-7a475...
146375	P_DM_Add_To_Box	P_TD_Products_Protected	prd	2023-06-28 09:00:49.842007 U...	2023-06-28 09:01:00.674428 U...	finished	ab20909d-d10b-432e-8094-7a475...
146376	P_DM_Add_To_Box	P_TD_Star_Products	prd	2023-06-28 09:01:05.109794 U...	2023-06-28 09:01:10.300286 U...	finished	ab20909d-d10b-432e-8094-7a475...
146377	P_DM_Add_To_Box	P_TD_Products	prd	2023-06-28 09:01:30.485583 U...	2023-06-28 09:01:37.733833 U...	finished	ab20909d-d10b-432e-8094-7a475...
146378	P_BR_Wave_Monitoring	P_TD_Events_Timeline	prd	2023-06-28 15:03:23.123543 U...	2023-06-28 15:13:57.998867 U...	finished	c8ea4550-1737-4f03-af6f-cf60db2...
146379	P_BR_Wave_Monitoring	P_TD_Subscription_History	prd	2023-06-28 15:00:50.312445 U...	2023-06-28 15:03:20.662774 U...	finished	c8ea4550-1737-4f03-af6f-cf60db2...

Running Pipelines as Cloud Run Jobs

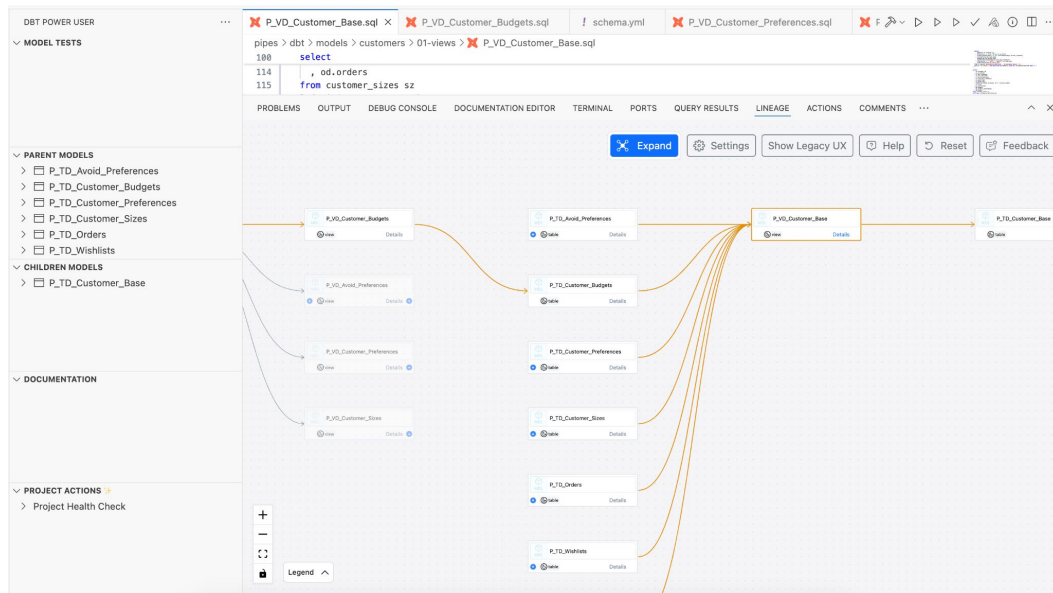
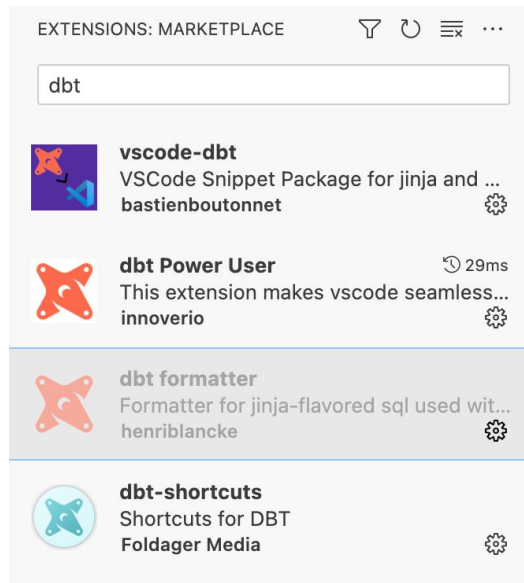
Cloud Run jobs - Used to run code that performs work (a job) and quits when the work is done

- Uses **same container** as Cloud Run service
- Flows can be specified as **arguments** to the **Docker** image
- Suitable for **long-running jobs**
- **Resources** can be specified for each job

```
27 spec:
28   taskCount: 1
29   template:
30     spec:
31       containers:
32         - image: gcr.io/...
33           command:
34             - python
35             - jobs.py
36           args:
37             - flow_products
38       resources:
39         limits:
40           cpu: 2000m
41           memory: 8Gi
42       maxRetries: 1
43       timeoutSeconds: '1800'
44       serviceAccountName: prefect-runner@...
```

Bonus: VSCode extensions for better dev experience with DBT

- **vscode-dbt** - provides snippets for dbt and Jinja functions
- **dbt Power User** - lineage, query results, execution of models, autocomplete, etc
- **dbt-shortcuts** - shortcuts for dbt commands (run, compile, test, etc)
- **vscode-dbt-language** - shortcuts for dbt model & compiled sql file associations



Use case:

Import pivoted data from
Google spreadsheets into
Bigquery

A	B	C	D	E	F	G	H	I
Scenario_Name	Scenario_Tag	Scenario_Total_	Jan-2025	Feb-2025	Mar-2025	Apr-2025	May-2025	Jun-2025
Marketplace	S0							
Marketplace	S0							
Marketplace	S0							
Marketplace	S0							
Marketplace	S0							
Marketplace	S0							
Marketplace	S1							
Marketplace	S0							
Marketplace	S0							

Solution: Use DBT External Loader

- Install **dbt_external_tables** package
- Define spreadsheets as sources in yaml
- Jinja for the dynamic pivoting (optional)
- SQL model for the transformation (from wide to long format)

Advantages:

- Everything stays in DBT
- Handles schema changes from spreadsheet
- Easy to add data tests

Row	scenario_name	scenario_tag	scenario_tot...	updated_at	month	revenue
1	Marketplace	S0		2025-05-13	2025-09-01	
2	Marketplace	S0		2025-09-12	2026-02-01	
3	Marketplace	S0		2024-09-13	2025-09-01	
4	Marketplace	S0		2025-02-07	2026-04-01	
5	Marketplace	S0		2025-04-08	2026-05-01	
6	Marketplace	S0		2025-05-13	2025-11-01	
7	Marketplace	S0		2025-07-10	2026-11-01	
8	Marketplace	S0		2024-07-18	2025-06-01	
9	Marketplace	S0		2024-09-13	2025-02-01	
10	Marketplace	S0		2024-11-07	2025-01-01	
11	Marketplace	S0		2025-05-13	2025-10-01	
12	Marketplace	S0		2025-05-13	2026-04-01	
13	Marketplace	S0		2025-05-13	2026-01-01	
14	Marketplace	S1		2025-06-10	2026-11-01	
15	Marketplace	S0		2025-07-10	2025-02-01	
16	Marketplace	S0		2025-07-10	2025-09-01	
17	Marketplace	S0		2025-07-10	2025-10-01	
18	Marketplace	S0		2024-10-21	2025-02-01	

You, 3 weeks ago | 1 author (You)

```
name: google_sheets
description: >
  | This source contains data stored in Google Sheets that are manually maintained.
dataset: stage_google_sheet
loader: external_table
tags:
  - source
  - google_sheet
```

Generate model

```
description: > Marketplace budgets
```

```
options:
```

```
uris: ['https://docs.google.com
sheet_range: MarketPlace_Budget
skip leading rows: 1
```

pipes > dbt > models > sprint_run > 02-tables >  P_TD_Budgets_Marketplaces.sql

You, 3 weeks ago | 1 author (You) | Add documentation or tests | Datapilot Notebooks

`select * from unpivoted` You, 3 weeks ago • Move budget load into DBT .

36 +

Agenda



**Serverless Data
Pipelines with
dbt, Prefect and
Cloud Run**

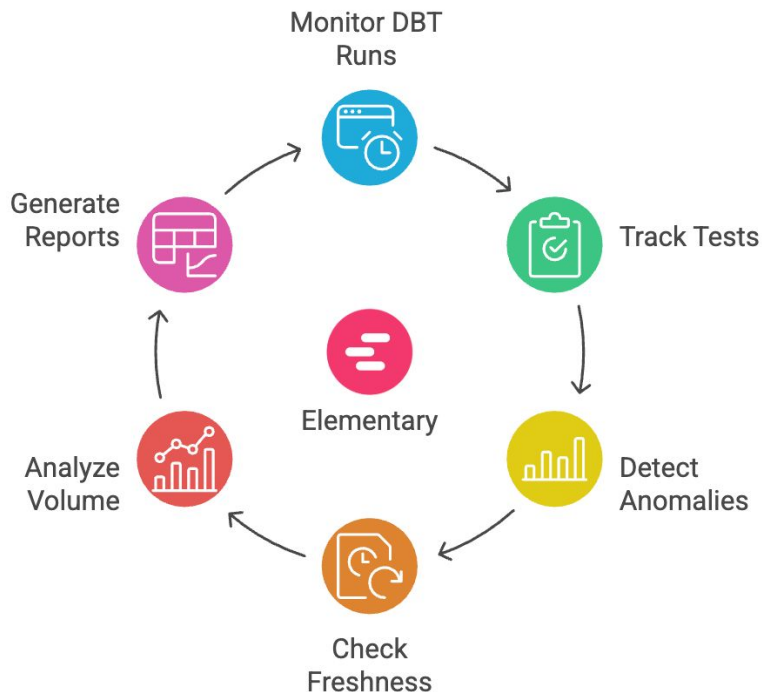


**Data
Observability
with
Elementary**



Q&A/Demo

Elementary: Observability for DBT



✓ Data Quality Checks

- Schema consistency (columns, types)
- Unique / Not Null / Accepted values

📈 Anomaly Detection

- Row count or volume anomalies
- Freshness / lateness detection
- Column value distribution shifts

🕒 Freshness & Timeliness

- Source freshness
- Pipeline run duration trends
- Data update frequency

📊 Reporting & Alerts

- Interactive HTML reports
- Slack / Email notifications
- Historical trend tracking
- Centralized metadata in warehouse

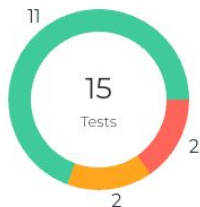
Dashboard

Filter by



Last 30 Days

Test results breakdown

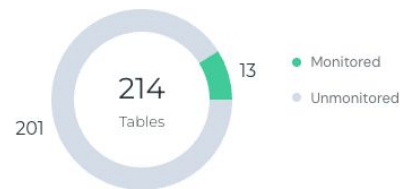


Tables health



- Passed
- Warning
- Failed
- Error

Monitored tables



- Monitored
- Unmonitored

Freshness

11
9 | 2 | 0

Volume

1
1 | 0 | 0

Schema changes

0
0 | 0 | 0

dbt tests

3
1 | 0 | 2

Anomalies

0
0 | 0 | 0

Model runs



- Success
- Error
- Skipped

Model run errors



- forecasting_engine_pipelines
- models
- analyses
- comparables
- demand_forecasts
- example
- preprocessing
- 01-views
- 02-tables
- size-breakdown
- sprint_run
- sup-export

Data Pipelines > Model Duration

Last 30 Days

65 Tables

Owners: -Tags: -Path: forecasting_engine_pipelines/models/preprocessing

Schema	Name	Last exec. time	Median exec. time	Ex time change rate	Results	Last run status	
p_tc_preprocessing	p_td_master_table_...	1.6 Mins	1.6 Mins	0.8%	28	Success	-

Model execution time and status over time

View in lineage

Copy link

Date	Execution Time (s)
Jun 12 10:08	85
Jun 13 10:08	100
Jun 14 10:07	90
Jun 15 10:07	85
Jun 16 10:08	100
Jun 17 10:08	85
Jun 18 10:08	110
Jun 19 10:08	85
Jun 20 10:07	95
Jun 21 10:07	90
Jun 22 10:07	130
Jun 23 10:07	95
Jun 24 10:07	95
Jun 25 10:07	110
Jun 26 10:08	95
Jun 27 10:08	130
Jun 28 10:08	130
Jun 29 10:07	95
Jun 30 10:07	90
Jul 1 10:08	100
Jul 2 10:08	95
Jul 3 10:08	100
Jul 4 10:07	95
Jul 5 10:07	90
Jul 6 10:08	85
Jul 7 10:08	95
Jul 8 10:08	90
Jul 9 10:08	95

p_br_size_breakdown	p_td_sb4_filtered_b...	41.9 Secs	38.6 Secs	8.5%	28	Success	+
p_tc_preprocessing	p_td_regularplus_pai...	36.5 Secs	4.5 Secs	711.1%	30	Success	+
p_tc_preprocessing	p_td_category_prod...	35.1 Secs	35.6 Secs	1.4%	28	Success	+

- forecasting_engine_pipelines
- models
- analyses
- comparables
- demand_forecasts
- example
- preprocessing
- 01-views
- 02-tables
- size-breakdown
- sprint_run
- sup-export
- sources
- tests

All Tables

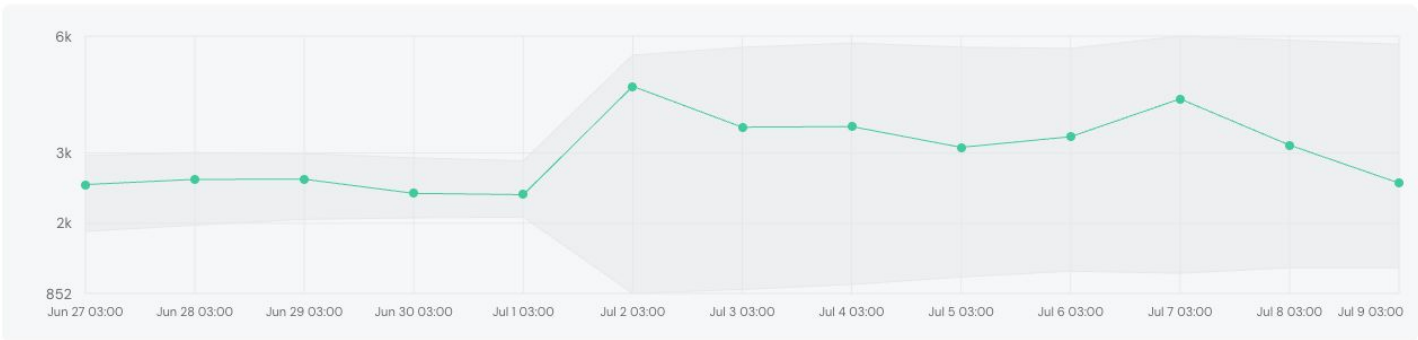
1022

Owners: -Tags: -Path: dbt_project/

Table nameColumn nameTest nameTest typeLast test runLast status

p_td_simples_sales_base-volume_anomaliesrow_count2025-07-09 11:00:59Passed

View test runsView in lineageCopy link



Description

Test Description:
Monitors the row count of your table over time.

Result

Result Description:
The last row_count value is 3210. The average for this metric is 3786.786.

Result Query:

Configuration

Test Name:volume_anomaliesTimeframe:1 dayTimestamp Column:Anomaly Threshold:

Elementary Deployment

- New route in FastAPI
- CLI command to generate Elementary report
- Report is saved into Google Storage
- Serve the report using Cloud Run with auth (IAP)

```
@app.get("/elementary")
def run_elementary(response: Response):
    try:
        slack_channel_name = SLACK_CHANNEL_ID if SLACK_CHANNEL_ID is not None else ""
        slack_bot_token = SLACK_BOT_TOKEN if SLACK_BOT_TOKEN is not None else ""
        result = subprocess.run([
            "edr", "send-report", "--gcs-bucket-name", "bdw-elementary-reports", "--bucket-file-path", "report.html",
            "--slack-channel-name", slack_channel_name, "--slack-token", slack_bot_token, "--update-bucket-website", "true",
            "--env", "prod", "--project-name", "demo_project", "--days-back", "30"
        ],
            capture_output=True,
            text=True,
            check=True
        )
        logging.info(result.stdout)
        response.status_code = status.HTTP_200_OK
        message = f"Elementary report created.\n{result.stdout}"
    except subprocess.CalledProcessError as e:
        logging.error(e.stderr)
        response.status_code = status.HTTP_500_INTERNAL_SERVER_ERROR
        message = f"Elementary report failed.\n{e.stderr}"
    return {"message": message}
```

Monitoring summary - am-forecasting-engine (prod)

This summary was generated with the following filters - timeframe: 30 days back

[View full report](#) 

 Passed: 6

 Errors: 0

 Skipped: 0

 Failed: 0

 Warning: 0

Recap

How everything works together

1. **Prefect** triggers DBT run
2. **DBT** executes transformations in BigQuery
3. **Elementary** analyzes and reports results
4. All running on **Cloud Run**

Benefits

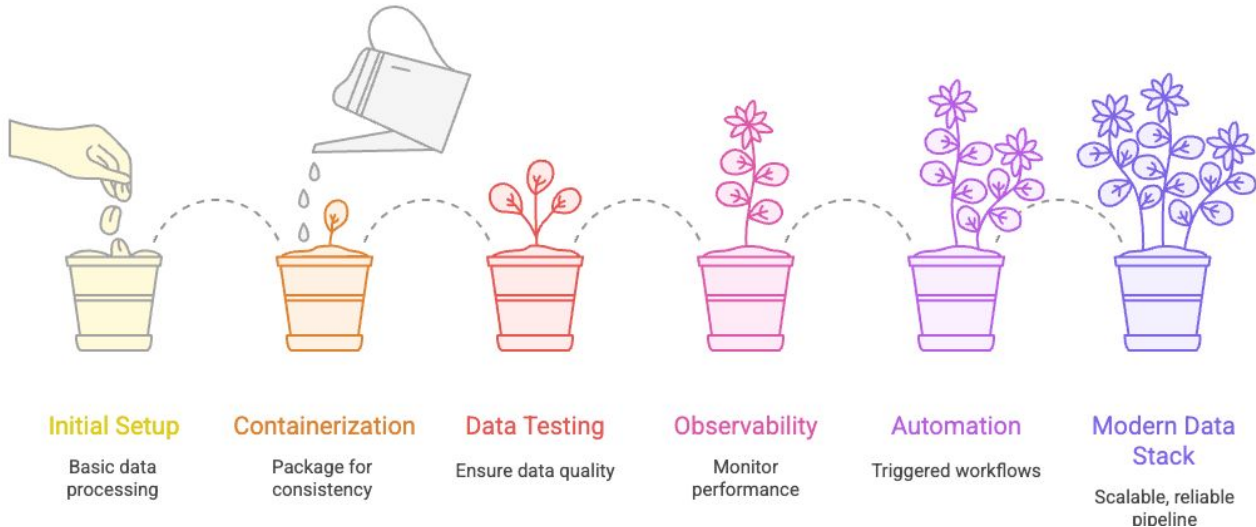
- **Serverless:** No infra, no idle cost
- **Scalable:** Runs in parallel, auto-scales
- **Observable:** Data quality + performance
- **Maintainable:** Versioned, testable SQL
- **Simple:** Modular, reusable workflows

Lessons Learned

- Start small, containerize early
- Add data tests from day one
- Integrate observability before scaling
- Automate triggers (Scheduler, CI/CD)

Key Takeaways

- **Modern stack:** DBT + Prefect + Elementary
- **Deployed serverlessly:** Cloud Run
- **Ideal for:** Analytics pipes, ELT, data quality



Q & A

Workshop repo:

<https://github.com/juma-adoreme/bdw-dbt-template>